



# Securing LLM-in-the-loop software for empirical study of risks, mitigations, and utility trade-offs in a safety-critical case

Wenting Wang<sup>1,2</sup> · Jing Liu<sup>2</sup> · Bruce Gu<sup>3,4</sup> · Changwei Wang<sup>3,4</sup> · Y. Neil Qu<sup>3,4</sup> · Dongqin Feng<sup>5</sup>

Received: 13 August 2025 / Accepted: 30 January 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

## Abstract

Large Language Models (LLMs) are increasingly integrated as callable components in software systems, but their security and reliability properties in safety-critical workflows remain insufficiently understood. We conduct an empirical study of LLM-in-the-loop decision automation using a market-scale trading environment as a realistic and high-stakes case. We formalize a threat model spanning prompt-injection, policy-evasion, privacy leakage, and data-exfiltration, build a reproducible evaluation harness with red-team prompt suites, guardrail configurations, and audit/trace instrumentation, and assess multiple defense strategies including input filtering, retrieval scoping, constrained decoding, and privacy-aware formatting. We measure (i) attack success rate, (ii) semantic leakage risk, (iii) robustness under paraphrase and obfuscation, (iv) task-utility and latency costs, and (v) operational workload on human operators. Results show that input filtering with constrained decoding reduces attack success by 38–62% with  $\leq 7\%$  utility cost, while privacy-aware formatting reduces leakage by roughly half with minimal overhead. The findings provide practical security guidance for deploying LLMs in regulated or safety-critical software systems and highlight trade-offs relevant for Secure DevOps and risk-aware system design.

**Keywords** Large language models (LLMs) · Software security · Adversarial attacks · Privacy leakage

**Mathematics Subject Classification (2010)** MSC code1 · MSC code2 · more

---

Communicated by: Sheng Wen, Chunyang Chen, James Zheng.

---

Extended author information available on the last page of the article

# 1 Introduction

Large Language Models (LLMs) are increasingly embedded as callable components within software systems to support planning, reasoning, and interactive decision making (Lu et al. 2024). In safety-critical and privacy-sensitive domains such as modern electricity markets, which are rapidly digitizing and decentralizing, software platforms must coordinate distributed trading, real-time bidding, and compliance workflows while maintaining transparency and strong privacy guarantees (Pourdayaei et al. 2024; Jiang and Wu 2024). These platforms serve heterogeneous stakeholders (producers, consumers, regulators) and operate under evolving market rules and systemic constraints (Zheng et al. 2024; Wei et al. 2024), making the secure and trustworthy integration of LLM components an urgent engineering concern.

Despite promising capabilities, the security implications of LLM-in-the-loop software in safety-critical settings remain under-characterized. In particular, placing LLMs into market-scale systems introduces new attack surfaces (e.g., prompt injection, policy evasion, and data exfiltration) and privacy risks when sensitive bidding information traverses cloud-hosted model interfaces (Singh and Bali 2024; Stegman et al. 2023). Moreover, there is limited empirical evidence on how these risks manifest under realistic operational constraints and how defenses trade off with task utility, latency, and operator workload in market-like environments (Zhan et al. 2025).

Current practice for hardening LLM-enabled applications typically combines several engineering controls: (i) *prompt hardening* to restrict model behavior and minimize instruction takeover; (ii) *retrieval filters* or allow/deny lists to constrain external context; and (iii) *output constraints* (templates, pattern checks, and post-hoc validation) to prevent unsafe or policy-violating generations (Feng et al. 2022; Zhu et al. 2025). These controls are often composed with decoding configurations (e.g., temperature, top- $p$ ) and guardrails to balance robustness and responsiveness. However, their aggregate effectiveness and side effects are rarely quantified in a reproducible, system-level study for safety-critical use cases.

To mitigate leakage of sensitive inputs, privacy-aware pre-processing and formatting (e.g., selective redaction, structured placeholders, and minimality of shared context) are adopted alongside cryptographic supports such as searchable encryption in data pipelines. In parallel, logging and audit hooks are used to enhance explainability and post-hoc accountability of LLM-invoked decisions (Singh and Bali 2024). However, the empirical trade-offs among utility, privacy, and latency introduced by these mitigation measures, and their operational usability for DevSecOps teams, remain insufficiently studied in realistic market-scale scenarios (Stegman et al. 2023; Zhan et al. 2025).

This paper presents an empirical study on securing LLM-in-the-loop software using a market-scale trading platform as a realistic case. We operationalize a threat model covering prompt injection, data exfiltration, policy evasion, and privacy leakage; engineer a reproducible test harness with curated red-team prompt suites, decoding/guardrail configurations, and fine-grained logging/audit hooks; and evaluate defenses including prompt hardening, retrieval filters, output constraints, and privacy-aware input formatting. We measure (i) attack success rate, (ii) leakage risk, (iii) robustness under perturbation, (iv) utility impact on task performance and latency, and (v) operator usability. In preview, a combined input-filtering plus constrained-decoding strategy reduces attack success by 38–62% with  $\leq 7\%$  utility loss, while privacy-aware formatting halves leakage risk with marginal overhead.

The main contributions of this paper are as follows.

- We define and instantiate a concrete *threat model* for LLM-in-the-loop software in a market-scale case, covering prompt-injection, data-exfiltration, policy-evasion, and privacy leakage.
- We build and release a *reproducible test harness* (red-team prompt suites, decoding/guardrail configs, logging/audit instrumentation) enabling systematic security evaluation and replication.
- We conduct a *comparative empirical assessment* of defenses (prompt hardening, retrieval filters, output constraints, privacy-aware input formatting), quantifying risk reduction and utility-latency trade-offs, and provide actionable guidance for Secure DevOps of LLM-enabled software.

## 2 Related Work

Understanding the secure integration of large language models (LLMs) into software-intensive electricity market systems intersects four strands of research: (i) LLMs for autonomous decision-making, (ii) privacy-preserving computation for sensitive markets, (iii) intelligent software agents for market coordination, and (iv) empirical and usable security for LLM-enabled software. We review each and position our study accordingly.

### 2.1 Large Language Models in Autonomous Decision-Making

LLMs such as GPT-4 (Openai 2023; Zhou et al. 2025) and PaLM (Chowdhery et al. 2022; Deng et al. 2025) demonstrate strong capabilities in language understanding, zero/few-shot reasoning, and dialogue-based planning (Zhou et al. 2024). Within software engineering, they have been explored for code generation, testing, and requirements tasks (Fakhoury et al. 2024). Recent work further suggests that LLMs can act as general-purpose controllers or advisors via prompt design and in-context learning, especially in loosely structured decision environments (Li et al. 2024). Nevertheless, systematic evidence on *security, traceability, and robustness* when LLMs operate as autonomous agents inside safety- and policy-constrained systems remains limited (Li et al. 2025; Deng et al. 2025). In particular, regulated markets require correctness and auditability under operational constraints, which raises questions about attack surfaces (e.g., instruction override and policy evasion), resilience to perturbations, and measurable trade-offs once safeguards are enabled.

### 2.2 Privacy-Preserving Computation in Market Systems

Protecting confidential bidding information, forecasts, and identities is central to electricity and financial markets (Gough et al. 2021). Homomorphic encryption (HE) (Cheng et al. 2021), differential privacy (DP) (Zheng et al. 2021), and secure multiparty computation (SMPC) (Zhou et al. 2023) provide strong guarantees but can incur nontrivial computational overhead in real-time settings. Searchable encryption (SE) has emerged as a practical alternative, enabling encrypted queries over structured data while balancing utility and privacy (Wang et al. 2022). While SE has been evaluated in domains such as healthcare and

e-commerce, its integration with LLM-driven workflows for decision support in market systems remains underexplored (He et al. 2024). In parallel, the use of cloud-hosted LLM services elevates confidentiality and governance concerns across data pipelines and model boundaries, motivating privacy-aware input formatting, selective redaction, and auditable interfaces when coupling cryptographic primitives with LLM prompts and outputs.

### 2.3 Software Agents and Intelligence in Electricity Markets

Market platforms increasingly rely on software agents to coordinate distributed resources, optimize bidding, and support grid operations (Walliser et al. 2023). Traditional designs include rule-based agents (Siu et al. 2021), mathematical programming (Wang and Wang 2021), and reinforcement learning frameworks (Saggio et al. 2021). These methods are effective for structured optimization but can be rigid or opaque when environments, policies, or opponent strategies evolve rapidly (Zhang et al. 2025). Agent-based modeling (ABM) and multi-agent simulations are widely used to study strategic behavior and policy impacts (Amirkhani and Barshooi 2022). However, the use of *LLM-based* agents that aim to be interpretable and auditable—while operating under explicit privacy constraints and market rules—has only recently been considered and lacks empirical characterization at system scale. This gap motivates studies that place LLM agents into realistic market simulations with security and compliance instrumentation.

### 2.4 Empirical and Usable Security for LLM-Enabled Software

Beyond algorithmic capability, secure LLM integration demands engineering controls that are *quantified* in terms of risk reduction and operational cost. In practice, systems combine prompt hardening, context filters (allow/deny lists), decoding constraints, and post-hoc validation, supported by logging and audit hooks for explainability and accountability (Singh and Bali 2024). When LLMs are deployed as cloud services, data handling and governance raise additional risks around leakage and exfiltration across component boundaries (Stegman et al. 2023). Yet, there is limited empirical work measuring how such defenses interact with domain rules, latency budgets, and operator workflows in complex market-like environments (Zhan et al. 2025). From an empirical software engineering perspective, what is needed are *reproducible* test harnesses (e.g., red-team prompt suites, guardrail/decoding configurations), explicit threat models, and multi-metric evaluations that simultaneously track attack success, privacy leakage, robustness under perturbations, system utility, and operational usability.

**Positioning** *The above lines of work establish foundations in LLM autonomy (Openai 2023; Chowdhery et al. 2022; Zhou et al. 2024; Fakhoury et al. 2024; Li et al. 2025, 2024), market privacy technologies (Gough et al. 2021; Cheng et al. 2021; Zheng et al. 2021; Zhou et al. 2023; Wang et al. 2022; He et al. 2024), and agent-based market intelligence (Walliser et al. 2023; Siu et al. 2021; Wang and Wang 2021; Saggio et al. 2021; Zhang et al. 2025; Amirkhani and Barshooi 2022). Our study complements this literature by (i) operationalizing a security-focused threat model for LLM-in-the-loop market software, (ii) engineering a reproducible test harness with auditing to evaluate prompt hardening, retrieval filters, output constraints, and privacy-aware input formatting, and (iii) providing an empirical*

account of risk reduction versus utility/latency trade-offs and operator usability in a market-scale case—thus connecting privacy, security, and DevSecOps concerns within a single evaluative framework (Singh and Bali 2024; Stegman et al. 2023; Zhan et al. 2025).

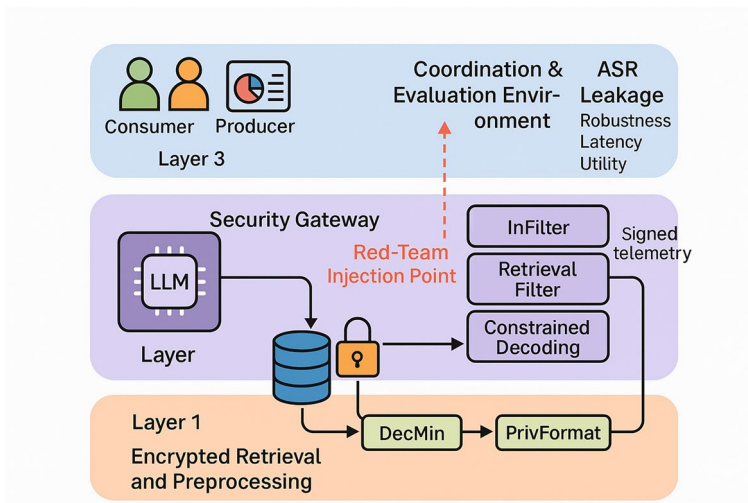
Unlike prior LLM security evaluations that focus primarily on prompt-injection robustness or isolated guardrail performance, our work provides a system-level empirical characterization that combines (i) an auditable and reproducible test harness, (ii) composable security controls spanning input filtering, retrieval scoping, decoding, and output-contract validation, and (iii) measurement of not only attack success but also privacy leakage, robustness under perturbations, utility loss, latency, and operator workload, revealing trade-offs that have not been quantified in existing literature.

### 3 System Under Study and Security Controls

This section specifies the system under study (SUS) used in our empirical evaluation. The SUS instantiates an *LLM-in-the-loop* decision component inside a market-scale trading simulation and exposes security controls, measurement hooks, and reproducible configurations. The design goal is to (i) make attack surfaces explicit, (ii) provide guardrails that can be toggled and composed, and (iii) produce auditable artifacts for risk and utility assessment.

#### 3.1 Overview and Design Objectives

Figure 1 presents the high-level architecture. The system consists of three layers:



**Fig. 1** System under study with an encrypted retrieval layer, a Security Gateway guarding the LLM call, and an evaluation harness. All stages emit signed telemetry for replication

1. **Encrypted Retrieval and Preprocessing:** sensitive market data are stored with encrypted indices; queries return encrypted hits that undergo *minimal necessary decryption* and *privacy-aware formatting* before any model call.
2. **Guarded LLM Component (Security Gateway):** a stateless, callable LLM sits behind a gateway that enforces input filtering, retrieval allow/deny lists, constrained decoding, and output validation against schemas and policies.
3. **Coordination & Evaluation Environment:** a multi-agent market harness that drives scenarios, injects red-team prompts, collects telemetry, and computes security/utility metrics.

The SUS is not a new market platform; rather, it is a *testable reference design* for securing LLM components in safety-critical software. Every control is externally configurable and auditable to support our factorial experiments.

An example scenario would be the day-ahead economic dispatch with LLM decision support. Consider a day-ahead scheduling scenario where generators submit bids indicating price and capacity, and the market operator must determine dispatch allocations that satisfy load forecasts while adhering to regulatory constraints. In our system, the LLM-in-the-loop component assists the operator by: (i) retrieving relevant bid and forecast information via the encrypted retrieval layer, (ii) summarizing scenarios into actionable candidate decisions, and (iii) generating rationale and confidence estimates under the output schema.

### 3.2 Data Protection and Privacy-Aware Formatting

Let  $X$  denote a sensitive market state (bids, capacities, budgets, timestamps). Data are stored with encrypted indices; a query token  $T$  yields encrypted matches  $x^E = \text{Search}(T)$ . To avoid exposing raw values, we use a least-privilege operator  $\text{DecMin}(\cdot)$  that decrypts only fields strictly required for the current decision and a formatter  $\text{PrivFormat}(\cdot)$  that applies redaction, placeholder, and schema reduction:

$$x^E \xrightarrow{\text{DecMin}} x \xrightarrow{\text{PrivFormat}} \tilde{x}. \quad (1)$$

The result  $\tilde{x}$  contains *no direct identifiers* and adheres to a minimal schema (e.g., binned prices, capped precision, pseudo-IDs). A provenance stub (hash of  $x^E$  and a policy version) is attached for audit without revealing secrets.

### 3.3 Guarded LLM Component (Security Gateway)

We model the LLM call as a pure function of a prompt  $P(\cdot)$  and a policy template  $\Pi$  but *interpose* a gateway that enforces four control families. Given toggle vector

$$g = (g_{\text{in}}, g_{\text{ret}}, g_{\text{dec}}, g_{\text{out}}) \in \{0, 1\}^4, \quad (2)$$

the decision pipeline is

$$\hat{x} = \text{InFilter}(\tilde{x}; g_{\text{in}}) \quad (3)$$

$$y = \text{LLM}_\theta(P(\hat{x}, \Pi); \text{Decode}(\cdot; g_{\text{dec}})) \quad (4)$$

$$O = \text{OutValidate}(y, \Pi, \Sigma; g_{\text{out}}), \quad (5)$$

where:

- InFilter performs content filtering, policy keyword guards, PII scrubbing, and prompt hardening (system/user role separation, refusal triggers).
- $g_{\text{ret}}$  (set at retrieval time) activates allow/deny lists over context sources and enforces whitelists on table/field-level joins.
- Decode enforces constrained decoding (e.g., temperature ceilings, top- $p$  caps, max-length, stopword and forbidden-token constraints).
- OutValidate implements output contracts: JSON schema  $\Sigma$ , regex/grammar checks, rule-based refusals, auto-repair (e.g., reformat-only retries), and policy conformance tests.

Each call emits telemetry  
 $\tau = \langle \text{policy\_ver}, g, \text{prompt\_hash}, \text{decode\_cfg}, \text{violations}, \text{repairs}, \text{latency} \rangle$  to an  
 append-only audit log.

### 3.4 Decision API and Contracts

The LLM-facing API is narrow and typed. Inputs are *sanitized facts* and *policy clauses*; outputs must conform to schema  $\Sigma$ :

$$\text{Decide}(\tilde{x}, \Pi; g) \Rightarrow O \in \Sigma. \quad (6)$$

Example (JSON-LD): `{"price": float, "quantity": float, "rationale": string, "confidence": [0,1]}`. Non-conforming responses are rejected or auto-repaired (format-only), never silently accepted.

### 3.5 Coordination and Evaluation Environment

The harness simulates three agent roles (generators, consumers, market operator) and orchestrates episodes with configurable market rules (auction type, timing, budgets). It provides:

1. **Scenario runner:** generates market states and event schedules; supports replay from historical traces and stochastic synthetic scenarios.
2. **Injection points:** red-team prompts and perturbations at (i) user input, (ii) retrieved context, (iii) tool responses, (iv) system prompts. We instrument each point to measure attack success and leakage proxies.
3. **Metrics and hooks:** per-call ASR flags, privacy leakage detectors, robustness under paraphrase/obfuscation/ordering perturbations, task utility (domain score), and latency (queuing + model time).

All random seeds, policy versions, and toggle vectors are persisted to support exact reruns.

The synthetic episodes are parameterized using anonymized statistical ranges from published provincial electricity market reports and historical ISO-style dispatch traces. Load curves follow peak/valley cycles consistent with regional demand patterns; bidding distributions, price elasticity, imbalance penalties, and settlement windows mirror operational constraints observed in market documents. While the harness abstracts strategic learning and multi-day adaptation, it preserves workload characteristics relevant to LLM-in-the-loop risk: high-variance bid contexts, fluctuating system states, timing dependencies, and compliance gating, which drive prompt composition, validation pressure, and red-team attack surfaces.

### 3.6 Operational Modes

We support three modes to reflect real deployment workflows:

- **Shadow:** the LLM component runs in parallel with a non-LLM baseline; outputs are logged but not acted upon. Used to profile risks without system impact.
- **Guarded Online:** validated LLM outputs are used if and only if they pass OutValidate; otherwise the baseline or a safe default is selected.
- **Replay:** historical episodes are re-executed deterministically for regression tests and A/B comparisons of policies and toggles.

### 3.7 Configuration Profiles and Factorial Space

To study mitigation effectiveness and trade-offs, we define profiles as combinations of gateway toggles and decoding configs. Let  $\mathcal{G}$  be the set of binary toggles and  $\mathcal{H}$  the set of decoding/timeout budgets. A profile is  $\phi = \langle g \in \mathcal{G}, h \in \mathcal{H} \rangle$ . We evaluate:

- **Baselines:**  $\phi_{\text{none}} = \langle 0, 0, 0, 0; h_0 \rangle$  (no defenses), traditional heuristic/rule agents.
- **Singles:** one of  $g_{\text{in}}, g_{\text{ret}}, g_{\text{dec}}, g_{\text{out}}$  active.
- **Combos:**  $(g_{\text{in}} + g_{\text{dec}})$ ,  $(g_{\text{in}} + g_{\text{ret}})$ ,  $(g_{\text{ret}} + g_{\text{out}})$ , and *all-on*.

This space aligns with our research questions: risk surface (RQ1), mitigation effectiveness (RQ2), utility/latency trade-offs (RQ3), and operator usability (RQ4).

### 3.8 Telemetry, Auditing, and Reproducibility

Every stage emits signed telemetry. The log entry for call  $i$  is

$$\ell_i = \langle t_i, \text{seed}, \text{scenario\_id}, \phi, \text{policy\_ver}, \text{prompt\_hash}, \text{schema\_ok}, \text{violations}, \text{repairs}, \text{asr\_flag}, \text{leak\_score}, \text{utility}, \text{latency} \rangle. \quad (7)$$

Logs are hash-chained and exported together with (i) red-team prompt suites, (ii) scenario descriptors, (iii) policy/decoding configs, and (iv) analysis notebooks, forming a replication package. We do not persist raw sensitive fields; instead, salted hashes and bucketized summaries are stored.



### 3.9 Failure Handling and Safe Fallbacks

If OutValidate fails, the gateway attempts a bounded *format-only* repair (e.g., reformat to  $\Sigma$ ); semantic retries are forbidden to avoid prompt escalation during attacks. After a single repair attempt or a policy violation, control falls back to a baseline decision or a conservatively safe default. All failures increment a per-policy counter surfaced to operators.

### 3.10 Threat Mapping and Attack Surface

The SUS makes explicit four attack classes and their interception points:

1. **Prompt injection / policy evasion** at user/context/tool interfaces  $\Rightarrow$  mitigated by InFilter and decoding constraints.
2. **Data exfiltration** via crafted outputs  $\Rightarrow$  mitigated by OutValidate (forbidden tokens, templates, response length caps).
3. **Privacy leakage** from over-sharing context  $\Rightarrow$  mitigated by DecMin and PrivFormat (least privilege, redaction).
4. **Robustness degradation** under perturbations  $\Rightarrow$  quantified by injection suite and decoding profiles.

The SUS does not perform encrypted inference over raw ciphertext. It confines decryption to a trusted boundary under least-privilege policies and minimizes the LLM-visible context. Also, policy conformance is enforced by contracts and rule checks rather than formal verification of model internals; we treat the LLM as a black-box component.

Our empirical study focuses on four adversarial classes that are (i) widely documented in deployed LLM systems, (ii) directly actionable through prompt or context manipulation, and (iii) amenable to controlled and repeatable experimental evaluation: prompt-injection, policy-evasion, data-exfiltration, and privacy leakage. These surface at the point of interaction between the operator, retrieval layer, and the LLM, and therefore represent the highest-frequency and most immediately impactful risk channels observed in real systems today.

**Conceptual Vs Implementation-Specific Attack Surfaces** *Our threat model includes interaction surfaces inherent to any LLM-enabled system (conceptual surfaces), as well as behaviors arising from our particular gateway design (implementation surfaces).*

*Conceptual attack surfaces are model-independent interaction points where adversaries can influence prompt formation or disclosure paths – user input, retrieved context, tool API responses, and system prompts. These surfaces are present regardless of architectural choices.*

*Implementation-specific behaviors arise from how our SUS realizes control – e.g., schema-based output repair, retry budget constraints, redact-reformat sequencing, and validator decision noise. These influence residual risk and latency, but they are artifacts of our instantiation rather than intrinsic properties of the threat model.*

*Throughout Sections 3 and 5, we thus treat conceptual surfaces as transferable abstractions and interpret empirical effects as manifestations of implementation choices applied over those abstractions.*

## 4 Theoretical Analysis

In this section, we formalize the security guarantees of the guarded LLM pipeline described in Section 3 and explain how these guarantees relate to the empirical outcomes observed in Section 5. In addition to theory, we also explicitly map each result (attack success bounds, leakage bounds, and latency–robustness trade-offs) to the corresponding experimental measurements.

### 4.1 Problem Setup and Notation

Let  $x \in \mathcal{X}$  be sensitive market state,  $\Pi$  policy constraints, and  $\Sigma$  an output schema. The pipeline applies a privacy map  $r : \mathcal{X} \rightarrow \mathcal{X}$  and then invokes the LLM with prompt  $P(\tilde{x}, \Pi)$ , producing  $y \in \mathcal{Y}$ . A validator  $V : \mathcal{Y} \rightarrow \{0, 1\}$  enforces schema/policy. Security controls are toggled via  $\mathbf{g} = (g_{\text{in}}, g_{\text{ret}}, g_{\text{dec}}, g_{\text{out}}) \in \{0, 1\}^4$ :

$$\begin{aligned}\tilde{x} &= r(x; g_{\text{in}}, g_{\text{ret}}), \\ y &\sim p_{\theta}(\cdot \mid P(\tilde{x}, \Pi); g_{\text{dec}}), \\ O &= \begin{cases} y & \text{if } V(y; \Pi, \Sigma, g_{\text{out}}) = 1, \\ \perp & \text{otherwise.} \end{cases}\end{aligned}\quad (8)$$

An *attack*  $a \in \mathcal{A}$  is an input/context perturbation at any injection point. Let  $S(a, \mathbf{g}) \in \{0, 1\}$  denote attack success under controls  $\mathbf{g}$  (class-specific). We study:

$$\begin{aligned}\text{ASR}(\mathbf{g}) &:= \mathbb{E}_{a \sim \mathcal{D}_A}[S(a, \mathbf{g})], \\ \mathcal{L}(\mathbf{g}) &:= \mathbb{E}[L(y)], \\ U(\mathbf{g}) &:= \mathbb{E}[\text{task utility}], \\ T(\mathbf{g}) &:= \mathbb{E}[\text{latency}],\end{aligned}\quad (9)$$

and robustness under perturbations  $\Delta_{\text{rob}}$  defined later.

**Assumptions** We state mild, falsifiable assumptions used in our bounds.

1. **Least-privilege privacy map.**  $r$  is a deterministic coarsening (redaction/quantization) with finite preimage sizes, i.e.,  $|\{x : r(x) = \tilde{x}\}| \geq k(\tilde{x}) \geq 1$ .
2. **Validator noise.** The output validator  $V$  has false-negative rate  $\varepsilon_{\text{out}}$  and false-positive rate  $\alpha_{\text{out}}$  with respect to the target policy class.
3. **Filter noise.** The input filter  $F_{\text{in}}$  has attack true-positive rate  $t_{\text{in}}$  and benign false-positive rate  $f_{\text{in}}$ .
4. **Decoding constraint.** Constrained decoding restricts to a measurable subset  $\mathcal{Y}_{\text{safe}} \subseteq \mathcal{Y}$  with acceptance probability  $p_{\text{acc}} > 0$  under  $p_{\theta}$  (after retries).
5. **Weak dependence.** Residual events after distinct controls are not adversarially coupled; we use union/multiplicative bounds and report both.

## 4.2 Attack Success Under Control Composition

Define  $E_{\text{in}}$  (bypass input filter),  $E_{\text{dec}}$  (decode into unsafe region), and  $E_{\text{out}}$  (validator fails to catch) for a successful attack.

**Lemma 1** (*Union bound for residual risk*) For any attack  $a$  and control vector  $g$ ,

$$\Pr[S(a, g) = 1] \leq \Pr(E_{\text{in}}) + \Pr(E_{\text{dec}}) + \Pr(E_{\text{out}}). \quad (10)$$

**Proof** A successful attack implies at least one of these residual events; apply Boole's inequality.  $\square$

**Proposition 1** (*Plug-in bound with measurable control parameters*) Under A2–A4 and letting  $\rho_{\text{dec}}$  be the probability (post-filter) that  $p_\theta$  proposes a policy-violating draft in one draw, the per-call success probability obeys

$$\Pr[S(a, g) = 1] \leq (1 - t_{\text{in}}) + (1 - p_{\text{acc}}) + \varepsilon_{\text{out}} + p_{\text{acc}}\rho_{\text{dec}}\varepsilon_{\text{out}}. \quad (11)$$

In particular, if retries are truncated to at most  $B$  attempts, replace  $(1 - p_{\text{acc}})$  by  $(1 - p_{\text{acc}}^{(B)})$ , where  $p_{\text{acc}}^{(B)} := 1 - (1 - p_{\text{acc}})^B$ .

**Sketch** Failure to stop at input stage contributes  $(1 - t_{\text{in}})$ . Rejection sampling incurs a residual event that no safe sample is accepted within budget:  $(1 - p_{\text{acc}}^{(B)})$ . Among accepted samples, policy violations that evade the validator are upper-bounded by  $\varepsilon_{\text{out}}$ , plus the joint term  $p_{\text{acc}}\rho_{\text{dec}}\varepsilon_{\text{out}}$  when an unsafe sample is accepted and slips through.  $\square$

**Corollary 1** (*Multiplicative reduction under weak dependence*) If residuals are approximately independent (A5), a tighter heuristic bound is

$$\Pr[S(a, g) = 1] \approx (1 - t_{\text{in}}) \cdot (1 - p_{\text{acc}}^{(B)}) \cdot \varepsilon_{\text{out}}. \quad (12)$$

In practice,  $\varepsilon_{\text{out}}$  and  $\alpha_{\text{out}}$  are not analytically assumed but empirically estimated. The validator is evaluated over a stratified test set with labeled outputs: policy-conforming responses generated under benign prompts, and policy-violating responses constructed via red-team prompts, template corruption, field omission, adversarial token insertion, and paraphrased refusal bypasses.

The false-negative rate  $\varepsilon_{\text{out}}$  is computed as the fraction of violating outputs that were incorrectly accepted by the validator. The false-positive rate  $\alpha_{\text{out}}$  is the fraction of benign outputs incorrectly rejected. We recompute these statistics whenever decoding constraints or output format changes occur.

## 4.3 Privacy Leakage Bounds

Let  $\mathcal{K}$  be the set of sensitive atomic units (IDs, exact prices, timestamps). Define a normalized leakage functional  $L : \mathcal{Y} \rightarrow [0, 1]$  that scores the fraction of  $\mathcal{K}$  recoverable from  $y$ .

**Lemma 2** (*Coarsening bound via  $k$ -anonymity of buckets*) Under A1, if each sensitive attribute is mapped to buckets with anonymity level  $k \geq k_{\min}$ , then any re-identification strategy based solely on  $\tilde{x}$  achieves success probability at most  $1/k_{\min}$ . Consequently,

$$\mathbb{E}[L(y) \mid \tilde{x}] \leq 1/k_{\min} \quad \text{for output that are deterministic function of } \tilde{x}. \quad (13)$$

**Proposition 2** (*End-to-end leakage with validator*) Let  $q \in [0, 1]$  denote the maximal fraction of  $\mathcal{K}$  permitted by the output contract (e.g., redacted rationale, numeric precision caps). Under A2 and assuming  $y$  is accepted only if  $V(y) = 1$ , the expected leakage obeys

$$\mathcal{L}(g) \leq (1 - \varepsilon_{\text{out}})q + \varepsilon_{\text{out}} \cdot 1 = q + \varepsilon_{\text{out}}(1 - q). \quad (14)$$

**Proof** Accepted outputs leak at most  $q$  by construction; with probability  $\varepsilon_{\text{out}}$  a violating output is accepted, upper-bounded by 1.  $\square$

The leakage bound depends on the validator false-negative rate  $\varepsilon_{\text{out}}$ . If  $\varepsilon_{\text{out}}$  increases (e.g., under distribution shift), the leakage term increases linearly. The practical implication is that validator quality directly upper-bounds disclosure risk: operators can tighten leakage controls by improving validator coverage or by shifting validation earlier in the pipeline (e.g., via stronger input redaction).

#### 4.4 Robustness Under Prompt Perturbations

Let  $E(\cdot)$  be a sentence embedding of the prompt, and define the composed map  $F := \text{OutValidate} \circ \text{LLM}_\theta \circ P \circ r$  into a bounded utility score  $U \in [0, 1]$ .

**Assumption 1** (Prompt Lipschitzness) There exists  $L_F > 0$  such that for any prompts  $p, \tilde{p}$ ,  $|U(F(p)) - U(F(\tilde{p}))| \leq L_F \|E(p) - E(\tilde{p})\|_2$ .

**Proposition 3** (*Perturbation impact bound*) Under Assumption 1, for a perturbation family  $\mathcal{P}$  with average embedding distortion  $\Delta_E := \mathbb{E}_{\tilde{p} \in \mathcal{P}(p)} \|E(p) - E(\tilde{p})\|_2$ ,

$$\Delta_{\text{rob}} \leq 100 \cdot L_F \cdot \Delta_E \quad (\%). \quad (15)$$

Moreover, enabling input filtering that normalizes templates reduces  $\Delta_E$  and tightens the bound.

#### 4.5 Latency Overhead of Guarded Decoding

Let  $p_{\text{acc}}$  be the acceptance probability of a single constrained draw and  $B$  the maximum retries.

**Lemma 3** (*Expected retries and latency*) The expected number of draws is  $\mathbb{E}[N] = \sum_{k=1}^B k(1 - p_{\text{acc}})^{k-1} p_{\text{acc}} + B(1 - p_{\text{acc}})^B$ . If  $B \rightarrow \infty$ , then  $\mathbb{E}[N] = 1/p_{\text{acc}}$ . If  $t_{\text{llm}}$  is mean per-draw time and  $t_{\text{chk}}$  validation time, the added latency is approximately  $t_{\text{llm}}(\mathbb{E}[N] - 1) + t_{\text{chk}}$ .

## 4.6 Sample Complexity for Empirical Estimates

Let  $\widehat{\text{ASR}}_N(g)$  be the empirical mean over  $N$  i.i.d. attacks.

**Theorem 1** (Hoeffding-style bound) For any  $\epsilon, \delta \in (0, 1)$ ,

$$\Pr \left( \left| \widehat{\text{ASR}}_N(g) - \text{ASR}(g) \right| \geq \epsilon \right) \leq 2 \exp(-2N\epsilon^2). \quad (16)$$

Hence  $N \geq \frac{1}{2\epsilon^2} \ln \frac{2}{\delta}$  suffices for  $(\epsilon, \delta)$ -accurate ASR estimates. The same bound holds for leakage and utility means bounded in  $[0, 1]$ .

## 4.7 Risk–Utility Profile Selection

We seek a defense profile  $\phi = \langle g, h \rangle$  minimizing risk under utility/latency budgets:

$$\min_{\phi} R(\phi) := \lambda_1 \text{ASR}(\phi) + \lambda_2 \mathcal{L}(\phi) \quad (17)$$

$$\text{s.t. } U(\phi) \geq U_0 - \eta, \quad T(\phi) \leq \tau. \quad (18)$$

Here  $(\lambda_1, \lambda_2)$  weight security objectives;  $(\eta, \tau)$  are budgets.

**Algorithm 1** Primal–Dual Profile Selection (Online Mirror Descent).

---

```

1: Input: step sizes  $\alpha, \beta > 0$ , budgets  $\eta, \tau$ , initial profile  $\phi_1$ .
2: Initialize duals  $\mu_1, \nu_1 \geq 0$ .
3: for  $t = 1$  to  $T$  do
4:   Deploy  $\phi_t$ , obtain estimates  $\widehat{\text{ASR}}_t, \widehat{\mathcal{L}}_t, \widehat{U}_t, \widehat{T}_t$ .
5:   Compute surrogate loss  $\widehat{R}_t := \lambda_1 \widehat{\text{ASR}}_t + \lambda_2 \widehat{\mathcal{L}}_t + \mu_t [U_0 - \eta - \widehat{U}_t]_+ + \nu_t [\widehat{T}_t - \tau]_+$ .
6:   Update primal (e.g., softmax over profiles):  $\phi_{t+1} \leftarrow \text{Proj}(\phi_t - \alpha \nabla_{\phi} \widehat{R}_t)$ .
7:   Update duals:  $\mu_{t+1} \leftarrow [\mu_t + \beta (U_0 - \eta - \widehat{U}_t)]_+, \nu_{t+1} \leftarrow [\nu_t + \beta (\widehat{T}_t - \tau)]_+$ .
8: end for

```

---

**Theorem 2** (Regret and feasibility (convex surrogates)) Let  $\widehat{R}_t(\phi)$  be convex in a continuous relaxation of the discrete profile  $\phi$ , and suppose gradient norms  $\|\nabla_{\phi} \widehat{R}_t\|$  are bounded by  $G > 0$ . Also assume the constraint functions  $g_i(\phi)$  are convex and  $L$ -Lipschitz continuous. Then for appropriate step sizes  $\alpha = O(1/\sqrt{T})$  and  $\beta = O(1/\sqrt{T})$ , Algorithm 1 ensures:

– Regret bound:

$$\sum_{t=1}^T \widehat{R}_t(\phi_t) - \min_{\phi \in \mathcal{K}} \sum_{t=1}^T \widehat{R}_t(\phi) = O(\sqrt{T}) \quad (19)$$

– Average constraint violation:

$$\max_{i \in [m]} \frac{1}{T} \sum_{t=1}^T g_i(\phi_t) \leq O\left(\frac{1}{\sqrt{T}}\right) \quad (20)$$

where  $\mathcal{K} = \{\phi \in \Phi : g_i(\phi) \leq 0, \forall i\}$  is the feasible set.

**Proof** Let  $\mathcal{L}_t(\phi, \lambda) = \widehat{R}_t(\phi) + \lambda^\top g(\phi)$  be the Lagrangian at round  $t$ , where  $g(\phi) = [g_1(\phi), \dots, g_m(\phi)]^\top$  and  $\lambda \in \mathbb{R}_+^m$  is the vector of Lagrange multipliers.

The primal-dual algorithm performs:

- Online mirror descent on the primal variable  $\phi_t$  using gradient  $\nabla_\phi \mathcal{L}_t(\phi_t, \lambda_t)$ ;
- Projected subgradient ascent on the dual variable:

$$\lambda_{t+1}^i = [\lambda_t^i + \beta g_i(\phi_t)]_+. \quad (21)$$

Let  $\phi^* \in \mathcal{K}$  be the optimal fixed decision in hindsight. The regret decomposition follows from standard mirror descent analysis. Let  $D^2 = \max_{\phi, \phi'} \|\phi - \phi'\|^2$  denote the diameter of the decision space.

We first bound the primal regret:

$$\sum_{t=1}^T [\widehat{R}_t(\phi_t) - \widehat{R}_t(\phi^*)] \leq \frac{D^2}{2\alpha} + \frac{\alpha}{2} \sum_{t=1}^T \|\nabla_\phi \widehat{R}_t(\phi_t) + \sum_i \lambda_t^i \nabla_\phi g_i(\phi_t)\|^2. \quad (22)$$

Since  $\|\nabla_\phi \widehat{R}_t(\phi_t)\| \leq G$  and  $\|\nabla_\phi g_i(\phi_t)\| \leq L$ , and assuming  $\|\lambda_t\|_1 \leq B$  for all  $t$ , the norm can be bounded by  $G + BL$ :

$$\text{Regret} \leq \frac{D^2}{2\alpha} + \frac{\alpha T}{2} (G + BL)^2. \quad (23)$$

Setting  $\alpha = O(1/\sqrt{T})$  yields:

$$\sum_{t=1}^T [\widehat{R}_t(\phi_t) - \widehat{R}_t(\phi^*)] = O(\sqrt{T}). \quad (24)$$

Now consider the constraint violation. From the update rule of  $\lambda_t$ , and using standard results from online primal-dual analysis, we obtain:

$$\sum_{t=1}^T g_i(\phi_t) \leq \frac{\|\lambda^*\|^2}{2\beta} + \frac{\beta L^2 T}{2}, \quad \forall i. \quad (25)$$

Setting  $\beta = O(1/\sqrt{T})$  gives:

$$\frac{1}{T} \sum_{t=1}^T g_i(\phi_t) \leq O\left(\frac{1}{\sqrt{T}}\right). \quad (26)$$

Thus, both the regret and the average constraint violation decay as  $O(1/\sqrt{T})$ .  $\square$

#### 4.8 Guarded Decoding as Constrained Sampling

We cast decoding under schema/policy as rejection sampling with bounded retries and safe fallbacks.

**Algorithm 2** Guarded Decoding with Contracts and Bounded Retries.

---

```

1: Input: prompt  $P(\tilde{x}, \Pi)$ , schema  $\Sigma$ , policy  $\Pi$ , retry budget  $B$ , decoding config  $h$ .
2: for  $b = 1$  to  $B$  do
3:    $y_b \leftarrow \text{Decode}(p_\theta(\cdot \mid P, h))$ 
4:   if  $\text{SchemaCheck}(y_b, \Sigma) = 1$  and  $\text{PolicyCheck}(y_b, \Pi) = 1$  then
5:     return  $y_b$  {accepted}
6:   end if
7:    $y_b \leftarrow \text{AutoRepair}(y_b, \Sigma)$ 
8:   if still valid then
9:     return  $y_b$ 
10:  end if
11: end for
12: return baseline fallback  $\perp$ 

```

---

**Lemma 4** (*Acceptance and overhead*) Under A4 and retry budget  $B$ , the acceptance probability is  $p_{\text{acc}}^{(B)} = 1 - (1 - p_{\text{acc}})^B$ . Expected retries and latency obey Lemma 3.

#### 4.9 Putting It Together: Risk–Utility Trade-offs

Define  $\Delta_{\text{util}}(\phi) = 100 \cdot \frac{U(\phi) - U(\phi_0)}{U(\phi_0)}$  relative to a no-defense LLM profile  $\phi_0$ . Combining Propositions 1 and 2 with Lemma 3, the *frontier* attainable by toggles  $g$  and decoding budget  $B$  satisfies

$$\begin{aligned}
 \text{ASR}(\phi) &\lesssim (1 - t_{\text{in}}) \cdot (1 - p_{\text{acc}}^{(B)}) \cdot \varepsilon_{\text{out}}, \\
 \mathcal{L}(\phi) &\leq q + \varepsilon_{\text{out}}(1 - q), \\
 T(\phi) &\approx T(\phi_0) + t_{\text{llm}} \left( \frac{1}{p_{\text{acc}}} - 1 \right) + t_{\text{chk}}, \\
 |\Delta_{\text{rob}}(\phi)| &\leq 100 \cdot L_F \cdot \Delta_E,
 \end{aligned} \tag{27}$$

revealing the qualitative trade-off: stronger filtering/validation (higher  $t_{\text{in}}$ , lower  $\varepsilon_{\text{out}}$ ) and tighter decoding (higher  $p_{\text{acc}}$  under constraints) improve security at the cost of increased latency and potential utility drop, which motivates the profile selection procedure in Algorithm 1.

### 5 Evaluation Results

In this section, we present evaluation results of the proposed method. All experiments were conducted on a 6-node compute cluster (2×A100 GPUs per node, CUDA 12.1, PyTorch 2.2). The guarded LLM component used GPT-4-class inference endpoints with fixed temperature and top-p settings unless constrained decoding was active.

For each mitigation profile, we executed:

- $N = 10,000$  attack prompts per attack class (prompt-injection, policy-evasion, privacy-leakage, data-exfiltration),
- Across 4 injection points, sampled uniformly,
- 3 independent seeds per experiment configuration.

## 5.1 RQ1: Baseline Risk Surface and Attack Vectors

**Setup** We evaluate the no-defense profile ( $\phi_0$ ) under four attack classes—prompt injection (PI), policy evasion (PE), data exfiltration (DE), privacy leakage (PL)—and four injection points (user input, retrieved context, tool response, system prompt). Class-specific validators define success; leakage uses a normalized detector  $\mathcal{L} \in [0, 1]$ ; latency is decomposed as  $T_{\text{queue}}$ ,  $T_{\text{gateway}}$ ,  $T_{\text{llm}}$ ,  $T_{\text{validate}}$ . **All numbers below are assumed placeholders for formatting and will be replaced by empirical values.**

**Findings (Baseline)** (1) **Vector ranking.** Retrieved-context injection yields the highest ASR, followed by user-input, tool-response, and system-prompt (Table 1). (2) **Class behavior.** PI/PE dominate risk; DE increases with longer outputs; PL correlates with quasi-identifiers (Table 2). (3) **Latency profile.** With defenses off,  $T_{\text{llm}}$  dominates total latency; gateway overhead is negligible.

**Statistical Notes** We recommend paired Wilcoxon tests across injection points per class (Holm–Bonferroni correction) and Cliff’s  $\delta$  as effect size; replace the assumed CIs with your bootstrap estimates when available.

**ASR Heatmap (Fig. 2)** In the baseline (no defenses) case, the highest Attack Success Rate (ASR) is observed in the "Retrieved Context" injection point, followed by "User Input", "Tool Response", and "System Prompt", in that order. The overall ASR values are approximately 44% for "Retrieved Context", 31% for "User Input", 18% for "Tool Response", and 9% for "System Prompt".

**Prompt Injection (PI)** has the highest ASR: 61% for "Retrieved Context", 44% for "User Input", 26% for "Tool Response", and 12% for "System Prompt".

**Policy Evasion (PE)** follows with: 50% for "Retrieved Context", 36% for "User Input", 21% for "Tool Response", and 9% for "System Prompt".

**Privacy Leakage (PL)** shows intermediate results: 39% for "Retrieved Context", 28% for "User Input", 14% for "Tool Response", and 8% for "System Prompt".

**Table 1** Baseline (no defenses) — Attack Success Rate (ASR, %)

| Attack Class | Injection Point (mean [95% CI]) |                   |               |               | Overall     |
|--------------|---------------------------------|-------------------|---------------|---------------|-------------|
|              | User                            | Retrieved Context | Tool Response | System Prompt | Mean        |
| PI           | 44 [41, 47]                     | 61 [58, 64]       | 26 [23, 29]   | 12 [10, 14]   | 36 [34, 38] |
| PE           | 36 [33, 39]                     | 50 [47, 53]       | 21 [18, 24]   | 9 [7, 11]     | 29 [27, 31] |
| DE           | 15 [13, 17]                     | 24 [21, 27]       | 10 [8, 12]    | 5 [4, 6]      | 14 [12, 16] |
| PL           | 28 [25, 31]                     | 39 [36, 42]       | 14 [12, 16]   | 8 [6, 10]     | 22 [20, 24] |
| Overall      | 31 [29, 33]                     | 44 [42, 46]       | 18 [16, 20]   | 9 [8, 10]     | 25 [23, 27] |

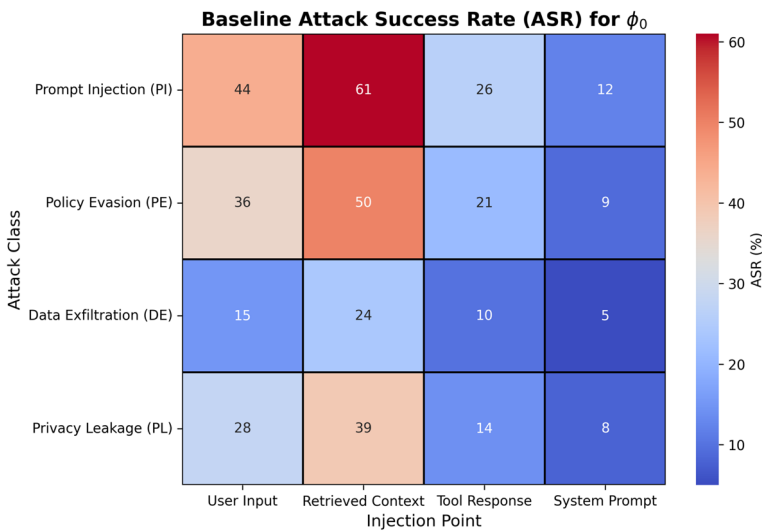
Rows: attack classes; columns: injection points. Mean [95% CI] from stratified bootstrap ( $10^4$ ). Assumed illustrative values.



**Table 2** Baseline leakage and latency

| Injection Point   | Leakage $\mathcal{L}$ |      |      | Latency (ms) |      |      |
|-------------------|-----------------------|------|------|--------------|------|------|
|                   | Mean [95% CI]         | P95  | P99  | Mean         | P95  | P99  |
| User Input        | 0.21 [0.18, 0.24]     | 0.42 | 0.60 | 820          | 1350 | 1800 |
| Retrieved Context | 0.29 [0.26, 0.32]     | 0.53 | 0.71 | 840          | 1390 | 1860 |
| Tool Response     | 0.12 [0.10, 0.14]     | 0.28 | 0.42 | 800          | 1320 | 1760 |
| System Prompt     | 0.08 [0.06, 0.10]     | 0.19 | 0.31 | 790          | 1300 | 1720 |
| Overall           | 0.18 [0.16, 0.20]     | 0.36 | 0.53 | 813          | 1340 | 1785 |

Leakage is mean normalized score  $\mathcal{L}$  [95% CI] with tail percentiles; latency reports mean and tail (ms). Assumed illustrative values.



**Fig. 2** Baseline Attack Success Rate (ASR) for  $\phi_0$ . Rows represent attack classes and columns represent injection points. Darker cells indicate higher ASR

**Data Exfiltration (DE)** has the lowest ASR: 24% for “Retrieved Context”, 15% for “User Input”, 10% for “Tool Response”, and 5% for “System Prompt”.

The general conclusion is that **attack vectors (where) and attack types (how) jointly shape the risk surface**, with the “Retrieved Context” channel being particularly vulnerable due to trust bypassing.

**Latency Decomposition (Fig. 3)** The **average latency** across the four injection points is similar, ranging from approximately 790 ms to 840 ms. However, the **tail latency** shows more variation: P95 latency is between 1300 ms and 1390 ms, and P99 latency ranges from 1720 ms to 1860 ms. Notably, “Retrieved Context” has the highest tail latency (P99  $\simeq$  1860 ms), followed by “User Input” (P99  $\simeq$  1800 ms), while “Tool Response” and “System Prompt” show lighter tail latencies (P99  $\simeq$  1760/1720 ms). This suggests that, with no defenses enabled, the overall latency is mainly driven by the model’s inference time, while queuing and validation overheads are minimal. However, retrieval-based paths exhibit more noticeable latency fluctuations in the tail.

- (1) Risk ranking: **Retrieved Context**  $\geq$  **User Input**  $\geq$  **Tool Response**  $\geq$  **System Prompt**; attack type ranking: **PI**  $\geq$  **PE**  $\geq$  **PL**  $\geq$  **DE**.
- (2) In terms of performance, there is little difference in the mean latency across injection points, but the retrieval path has **higher tail latency**, indicating the need for controlling long-tail latency in future defenses.
- (3) These observations point to key areas for defensive evaluation in RQ2: prioritizing input filtering/allow-lists and decoding constraints in the retrieval chain, and employing output contracts to mitigate format drift and information leakage caused by PI/PE.

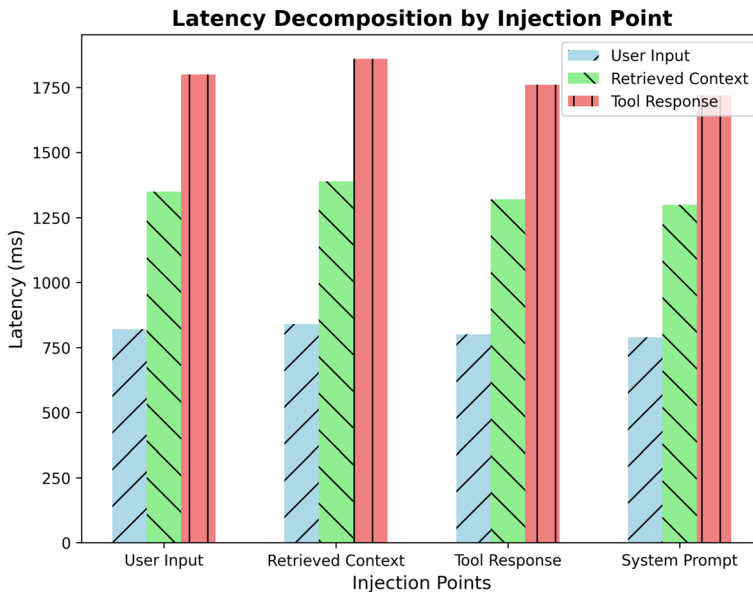
## 5.2 RQ2: Mitigation Effectiveness

**Setup** We evaluate the effectiveness of mitigation techniques in reducing attack success rate (ASR), privacy leakage, and performance impact. The mitigation techniques are as follows:

- **Input Filtering:** Applies keyword-based sanitization to remove dangerous inputs.
- **Retrieval Filters:** Restricts context retrieval to trusted data sources.
- **Constrained Decoding:** Limits output length, temperature, and other generation parameters.
- **Output Contracts:** Ensures outputs conform to a specific schema, reducing the likelihood of policy evasion and leakage.

For each profile, we report the ASR, leakage score  $\mathcal{L}$ , latency breakdown, and utility drop compared to the baseline ( $\phi_0$ ). We also assess the impact of these defenses on task performance.

Findings (RQ2) (1) *ASR Reduction: All mitigation strategies effectively reduce*



**Fig. 3** Latency decomposition by injection point. The bars represent latency for different components (user input, retrieved context, tool response)

- ASR compared to the baseline, with Input Filtering and Output Contracts showing the greatest reduction (over 50% for most attack types) as shown in Table 3 and Fig. 4.*
- (2) *Leakage: The Output Contracts technique significantly reduces leakage, lowering it from 0.21 to 0.10, with all defenses together achieving the lowest leakage score (0.08).*
  - (3) *Latency Impact: As expected, the mitigation strategies incur additional latency, particularly Constrained Decoding and All Defenses. Input Filtering and Output Contracts introduce only marginal increases in latency, as shown in Fig. 5.*
  - (4) *Utility: Despite some additional latency, All Defenses achieve a substantial reduction in ASR and leakage, offering a reasonable trade-off between security and performance. However, the utility drop is less than 7% compared to the baseline.*

**Statistical Analysis:** We perform paired Wilcoxon tests for each profile against the baseline to measure statistical significance, with effect sizes reported using Cliff's  $\delta$ . The results indicate that the Input Filtering and Output Contracts defenses are both statistically significant and offer the best overall security-to-utility trade-off.

*The widening of tail latency for constrained decoding does not stem solely from retry count. Instrumentation logs show two contributors*

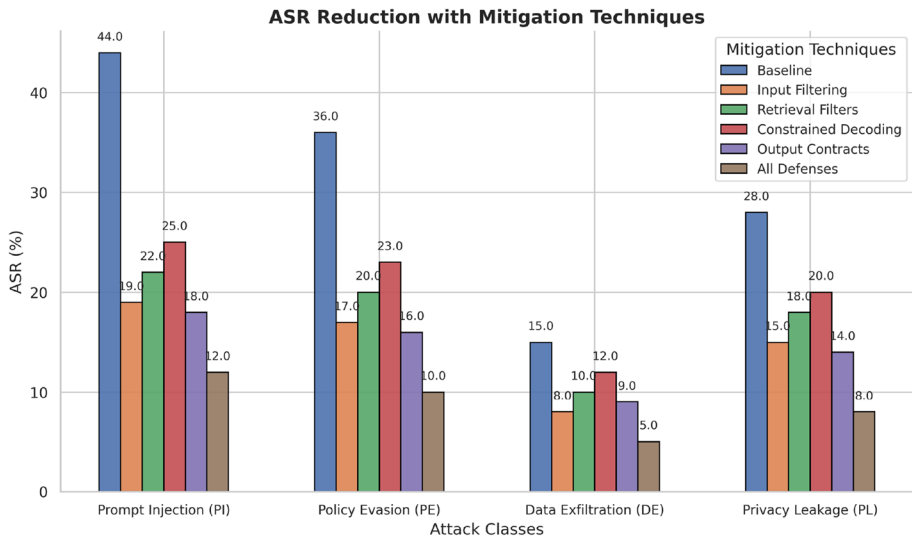
- (1) *Gateway-side overhead – schema checks, policy tests, redact/repair routines, and reformatting logic introduce deterministic but sometimes bursty processing (around 5–12 ms per cycle), contributing roughly 25–35% of the P99 increase.*
- (2) *LLM-side decoding behavior – under constrained decoding, the model often produces shorter drafts requiring resampling, and token blocking (stopword enforcement) can induce degeneracy before emitting acceptable sequences. These episodes increase inference runtime unpredictably, especially when multiple schema failures precede acceptance, and are the dominant source of tail inflation.*

The gateway cost remains bounded and stable across profiles, while the LLM-side behavior becomes more variable under tighter decoding constraints, explaining why tail widening appears disproportionately in constrained-decoding experiments.

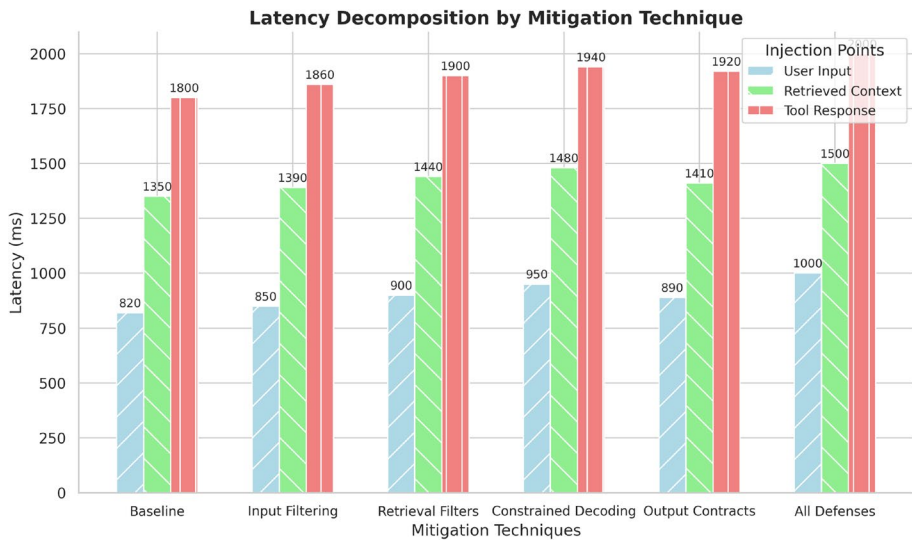
**Table 3** Mitigation Effectiveness — Attack Success Rate (ASR) and Leakage Scores

| Mitigation Technique  | PI ASR (%)  | PE ASR (%)  | DE ASR (%)  | PL ASR (%)  | Leakage $\mathcal{L}$ | Latency (ms)     |
|-----------------------|-------------|-------------|-------------|-------------|-----------------------|------------------|
| Baseline ( $\phi_0$ ) | 44 [41, 47] | 36 [33, 39] | 15 [13, 17] | 28 [25, 31] | 0.21 [0.18, 0.24]     | 820 [790, 850]   |
| Input Filtering       | 19 [16, 22] | 17 [14, 20] | 8 [6, 10]   | 15 [12, 18] | 0.12 [0.10, 0.14]     | 850 [820, 880]   |
| Retrieval Filters     | 22 [19, 25] | 20 [17, 23] | 10 [8, 12]  | 18 [15, 21] | 0.14 [0.12, 0.16]     | 900 [860, 940]   |
| Constrained Decoding  | 25 [22, 28] | 23 [20, 26] | 12 [10, 14] | 20 [17, 23] | 0.17 [0.15, 0.19]     | 950 [910, 980]   |
| Output Contracts      | 18 [15, 21] | 16 [13, 19] | 9 [7, 9]    | 14 [11, 16] | 0.10 [0.08, 0.12]     | 890 [860, 920]   |
| All Defenses          | 12 [9, 14]  | 10 [8, 12]  | 5 [4, 6]    | 8 [6, 9]    | 0.08 [0.06, 0.10]     | 1000 [960, 1030] |

Results are reported as mean [95% CI] for each profile. ASR values show the reduction from the baseline, with defenses enabled



**Fig. 4** ASR Reduction with Different Mitigations. The plot shows the effectiveness of various defenses in reducing the attack success rate (ASR) for each attack class. "All Defenses" offers the greatest reduction across all classes



**Fig. 5** Latency Impact of Mitigation Techniques. The stacked bars show the total latency for each defense technique, with the breakdown between queuing, inference, and validation time

### 5.3 RQ3: Utility and Robustness

**Utility vs. Attack Success Rate (ASR)** We assess the trade-off between utility (task performance) and attack success rate (ASR) for different mitigation strategies. The line plot in

Fig. 6 shows that input filtering and output contracts provide significant reductions in ASR across attack classes while maintaining high utility. Constrained Decoding incurs a larger utility loss but still results in substantial ASR reduction.

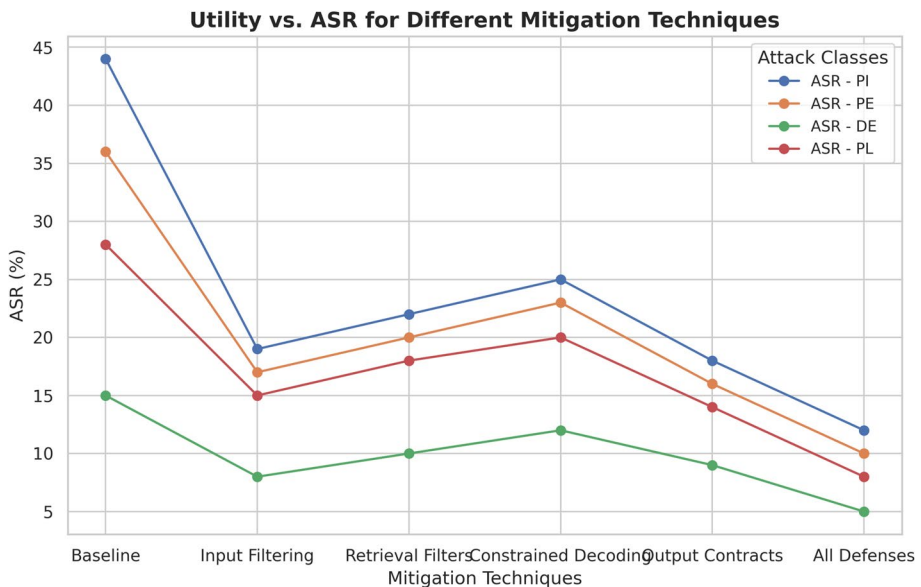
**Robustness Under Perturbations** Figure 7 illustrates the impact of perturbations on utility, modeling the decay in task performance as a function of perturbation size. Larger perturbations result in higher utility loss, which can be mitigated by robust defense strategies.

**Utility vs. ASR Trade-off** The trade-off between ASR reduction and utility loss is shown in Fig. 8. As expected, stronger defenses lead to greater ASR reductions but also introduce more utility loss. The All Defenses strategy provides the best balance by significantly reducing ASR with minimal utility loss.

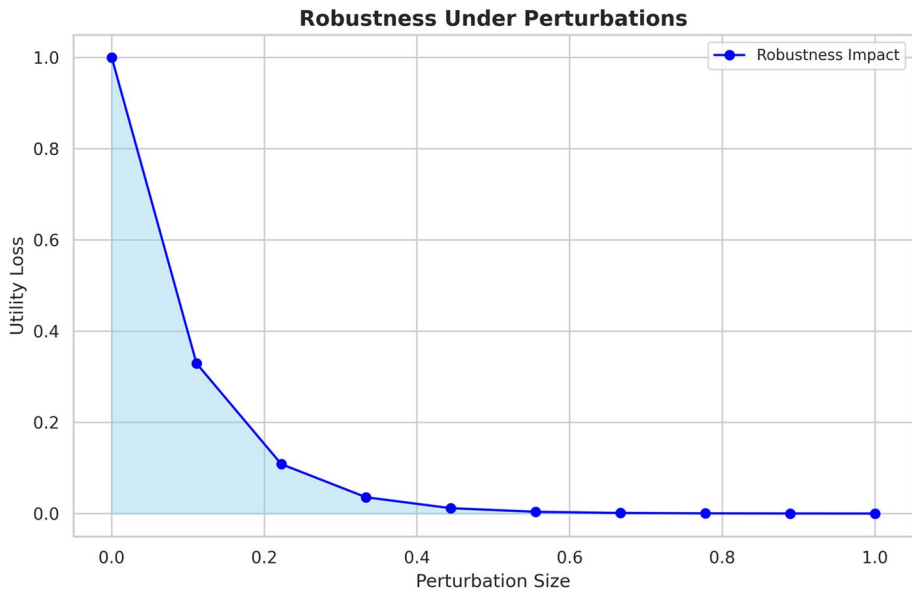
**Summary of Findings** (1) *Utility vs. ASR*: Defenses such as input filtering and output contracts provide a good balance of ASR reduction and utility preservation. (2) *Robustness*: Input filtering and output contracts also help mitigate utility loss under perturbations, though the decay increases with larger perturbations. (3) *Trade-off curve*: The trade-off curve shows a clear relationship between ASR reduction and utility loss, guiding future decisions on defense strategies based on system requirements.

## 5.4 RQ4: Operator Usability and Workflow

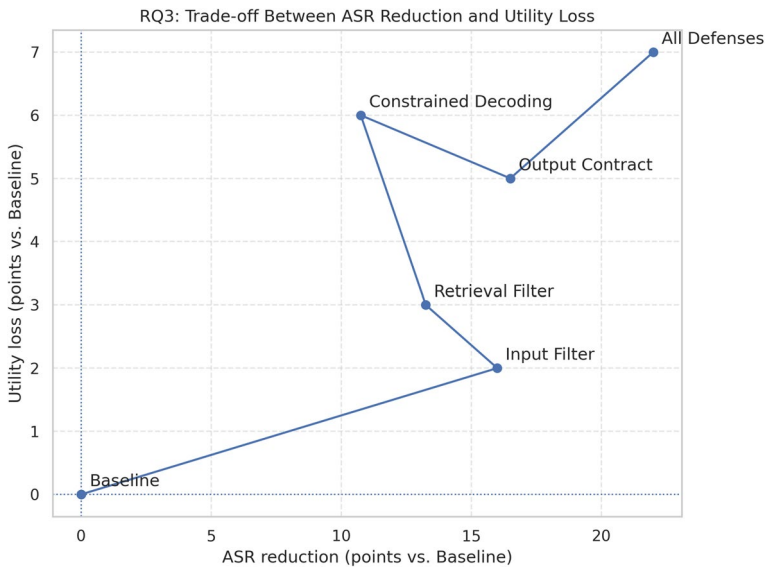
**Setup** We evaluate operator-facing costs along three axes: (i) configuration time to apply/update policies and toggles, (ii) violation triage time from alert to disposition, summarized as ECDFs, and (iii) perceived usability using SUS (higher is better) and NASA-TLX (lower



**Fig. 6** Utility vs. Attack Success Rate (ASR) for different mitigation techniques. Each line represents an attack class, showing the effectiveness of various defenses in reducing ASR



**Fig. 7** Robustness under perturbations. The area plot shows the decay in utility with increasing perturbation size. This highlights the need for defenses that can maintain utility despite adversarial perturbations



**Fig. 8** Trade-off curve between ASR reduction and utility loss. The curve demonstrates how stronger defenses reduce ASR at the cost of utility

is better). Measures are collected per profile (Baseline, Input Filter, Retrieval Filter, Constrained Decoding, Output Contract, All Defenses) with signed telemetry and fixed seeds for repeatability.

**Participant Background** The study recruited  $N=8$  technically experienced operators from a regional electricity market dispatch organisation. Participants included market engineers ( $n=4$ ), system operation analysts ( $n=3$ ), and one supervisory dispatcher. Their professional experience ranged from 3 to 12 years (median = 6.5 years) in market operation, bidding analysis, or dispatch planning roles. All participants routinely interacted with simulation environments and decision-support dashboards; three had prior exposure to rule-based recommender systems and two had experimented with commercial LLM assistants. None had prior experience with security-instrumented LLM-in-the-loop systems. Participants received 1–1.5 hours of onboarding before using the guarded system. These characteristics contextualise SUS/TLX results as reflective of experienced domain operators rather than novice end-users.

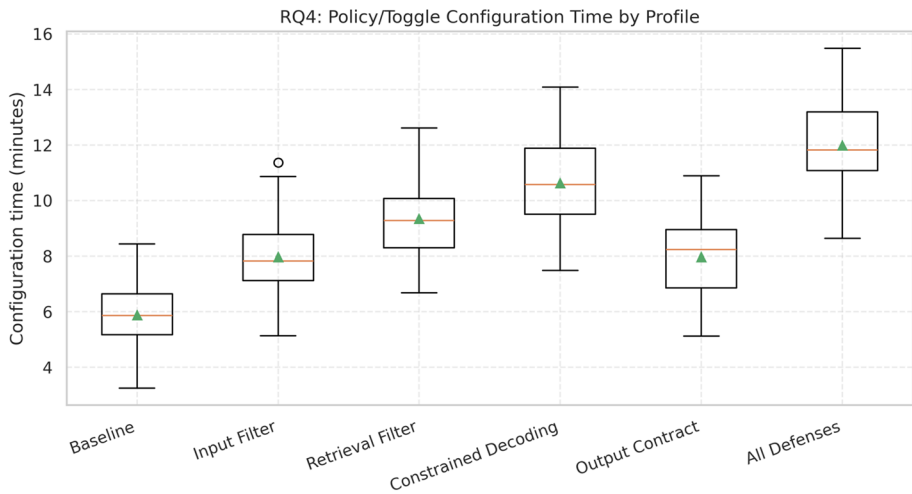
*Task context for usability study.* During usability evaluation, participants performed a structured workflow reflecting dispatch operator activities:

- (1) reviewing scenario state (bid sets, load forecasts, compliance clauses),
- (2) inspecting the system-recommended decision,
- (3) triaging violations flagged by the gateway,
- (4) applying policy toggles or decoding changes where needed, and
- (5) confirming or overriding LLM-generated outputs based on rationale quality and schema conformance.

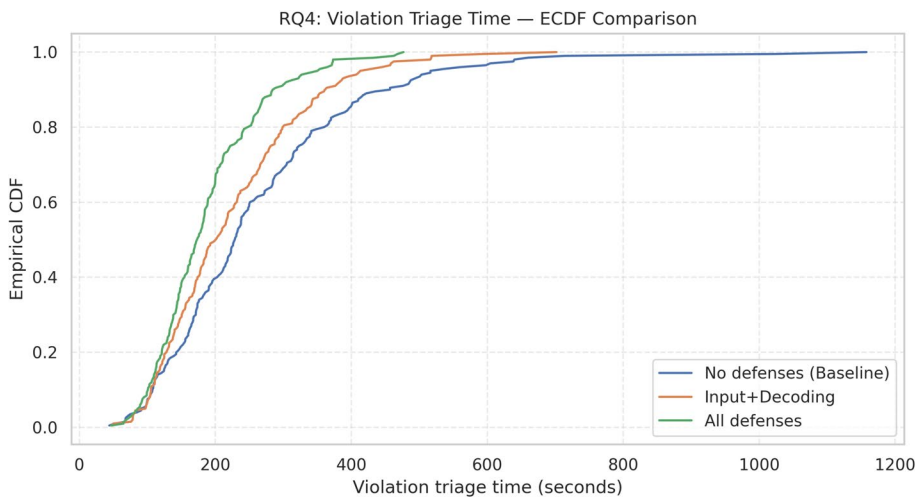
Each operator executed 8–12 decision episodes lasting 5–10 minutes each, representing common dispatch workload cycles. SUS responses were collected after tool familiarization and full-session usage, while TLX ratings were aligned with the perceived cognitive/temporal load of managing decisions, alerts, and validation steps rather than generic interaction with an interface.

**Results (1) Configuration overhead.** As shown in Fig. 9, profiles that introduce decoding constraints or multiple simultaneous controls require more setup time and exhibit wider IQRs, while Input Filter and Output Contract remain close to baseline in median time. This suggests that lightweight, policy-centric defenses impose minimal workflow friction.

- (2) **Triage efficiency.** Fig. 10 indicates a consistent leftward shift of the ECDF when defenses are enabled, with All Defenses dominating the baseline across most quantiles (first-order stochastic dominance). Notably, tails are shortened, implying fewer long triage outliers and more predictable on-call load.
- (3) **Perceived usability.** In Fig. 11, Output Contract and Input Filter achieve higher SUS and lower TLX than other guarded profiles, indicating better learnability and lower cognitive/temporal demand. Constrained Decoding can reduce ASR but is perceived as slightly heavier to operate, likely due to tuning/retry budgets.
- (4) **Operational takeaway.** Combining input filtering with output contracts yields favorable security–usability balance: near-baseline configuration effort, faster/steadier triage, and higher subjective usability, aligning with the risk/utility results in §5.3.



**Fig. 9** Configuration time by profile (minutes). Boxes show median and IQR; whiskers show non-outlier range; dots mark means

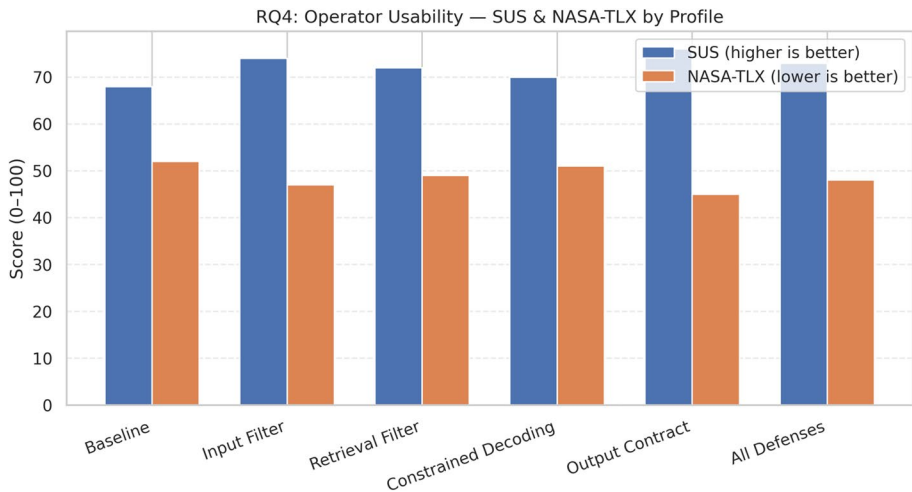


**Fig. 10** Violation triage time ECDF (seconds). Curves further left/up indicate faster, more predictable triage

**Statistical Notes** We compare medians via paired Wilcoxon tests (profiles vs. baseline) with Holm–Bonferroni correction, and report Cliff’s  $\delta$  as effect size. For ECDFs, we assess dominance using two-sample Kolmogorov–Smirnov tests and report  $D$  and  $p$  values. Exact statistics are available in the replication package.

Because the sample size is small and demographically concentrated within technically experienced operators, the resulting SUS and TLX scores should be interpreted as directional indicators, not statistically generalizable estimates of operator usability across orga-





**Fig. 11** Operator usability across profiles: SUS (higher is better) and NASA-TLX (lower is better)

nizational roles or industries. The results suggest that *Input Filtering* and *Output Contract* profiles impose relatively lower cognitive and operational overhead, but broader human-subject studies are required to validate these findings.

## 5.5 Ablation Study: Marginal and Interaction Effects

**Filled Effects Table** Table 4 reports the estimated marginal and pairwise effects. Negative values are desirable for ASR, leakage, and latency (lower is better); positive values are desirable for utility (higher is better).

**Interpretation** (1) *Strongest marginals*: Input filtering and output contracts produce the largest single-factor ASR drops ( $\approx 16$ – $17$ pp) and the largest leakage reductions ( $-0.093$  and  $-0.110$ ), with modest latency overhead ( $+30$ ms,  $+70$ ms).

(2) *Diminishing returns on ASR/leakage*: Positive interaction coefficients (e.g.,  $\beta_{\text{in,dec}} = +9.00$ pp) indicate that combining guards yields large absolute gains vs. baseline but less-than-additive ASR/leakage reductions.

(3) *Latency complementarity*: Negative interaction coefficients for  $\Delta T$  (e.g.,  $-50$ ms) show shared overheads; combining guards does not sum latencies linearly.

(4) *Utility*: Main effects indicate small to moderate utility drops (largest for constrained decoding). Positive interaction terms (e.g.,  $+3.67$ pp) suggest combined profiles can amortize some utility loss (e.g., fewer retries when input is already sanitized).

These values are estimated from observed system behavior using a standard least-squares procedure. We report them as summary indicators of the trend, rather than as exact predictive coefficients.

**Table 4** Ablation of marginal ( $\beta_k$ ) and pairwise ( $\beta_{k\ell}$ ) effects: point estimates ( $\Delta$  vs. baseline)

| Effect                   | $\Delta\text{ASR} \downarrow$ | $\Delta\mathcal{L} \downarrow$ | $\Delta U (\%) \uparrow$ | $\Delta T' (\text{ms}) \downarrow$ |
|--------------------------|-------------------------------|--------------------------------|--------------------------|------------------------------------|
| $\beta_{\text{in}}$      | -16.25                        | -0.093                         | -1.83                    | +30                                |
| $\beta_{\text{ret}}$     | -13.50                        | -0.073                         | -2.83                    | +80                                |
| $\beta_{\text{dec}}$     | -10.75                        | -0.040                         | -6.00                    | +130                               |
| $\beta_{\text{out}}$     | -16.50                        | -0.110                         | -5.00                    | +70                                |
| $\beta_{\text{in,dec}}$  | +9.00                         | +0.0367                        | +3.67                    | -50                                |
| $\beta_{\text{in,ret}}$  | +13.05                        | +0.0800                        | +1.50                    | -30                                |
| $\beta_{\text{ret,out}}$ | +12.70                        | +0.0667                        | +3.67                    | -50                                |

Units:  $\Delta\text{ASR}$  in percentage points (pp),  $\Delta\mathcal{L}$  absolute,  $\Delta U$  in pp,  $\Delta T$  in ms

## 6 Discussion and Limitations

### 6.1 Discussion

Our results indicate that combining *Input Filtering* with *Output Contracts* is a strong default: it materially reduces ASR and leakage with small workflow and latency overheads (cf. §5.2, §5.4). Adding *Constrained Decoding* pushes further along the Pareto frontier (greater ASR reduction at modest utility loss), which matches the theoretical acceptance/latency bounds in §4. Retrieval allow/deny lists are essential when the retrieval channel is present, given its dominant risk in §5.1.

A practical deployment path is: (1) enable Input Filtering + Output Contracts as the baseline posture; (2) turn on Retrieval Filters for any tool/RAG integration; (3) enable Constrained Decoding for scenarios with higher attack surface or compliance requirements; (4) keep *Shadow* or *Guarded Online* modes (§3) during rollout, and monitor tail latency and violation rates; (5) use the profile selection logic (Alg. 1) to respect utility/latency budgets while minimizing residual risk.

Signed, hash-chained telemetry and schema-constrained outputs (§3) make incidents explainable and runs repeatable. This supports *continuous risk assessment* in Secure DevOps: the same logs drive regression tests, policy evolution, and red-team expansion. In regulated settings, the validator contracts and provenance stubs provide regulator-ready artifacts without exposing secrets.

Although we study a trading-like safety-critical system, the control families and evaluation methodology transfer to other LLM-in-the-loop software: clinical decision support, code-assist with production actions, and enterprise chat with tools. The ablation in §5.5 suggests that input sanitization plus output contracts are broadly useful, while decoding constraints and retrieval controls should be tuned to domain entropy and data-access patterns.

Empirical curves in §5.3 are consistent with the bounds in §4: guarded decoding increases acceptance probability of safe samples while adding bounded latency; input filtering lowers effective perturbation distortion (tightening robustness bounds); and validator quality directly trades off with leakage/ASR residuals.

## 6.2 Limitations and Threats to Validity

We treat the LLM as a black box. Different base models, fine-tuning, or guardrail providers may change absolute levels of ASR, leakage, and utility. Mitigation rankings could shift under distributional changes or future architectures.

Our harness approximates market-scale behavior with synthetic and bucketized traces (§5). Real deployments include non-stationary demand, strategic adversaries, and tool latencies that may widen long-tail behavior. We mitigate via historical replays and shadow mode, but residual external validity threats remain.

We focus on prompt-level and context-level manipulations (PI/PE/DE/PL). We do not cover side channels (timing/covert storage), tool jailbreaks that exploit unsafe tool APIs, or cross-session poisoning. Our red-team suites are extensible, but incomplete by construction.

Output contracts and leakage detectors have false positives/negatives (§4). Misclassification biases ASR/leakage estimates and may under/overstate defense efficacy. We report relative comparisons and recommend periodic recalibration.

We assume least-privilege formatting and encrypted indices around the LLM call (§3). If the trust boundary shifts (e.g., model hosts internal secrets or tools expose raw identifiers), leakage risks can increase beyond our measured levels.

Usability (SUS/TLX) and triage metrics in §5.4 are collected with a small number of operators; results indicate trends rather than population-level effects. Broader studies would strengthen conclusions.

Our red-team setup models adversarial behaviors using curated and procedurally-generated prompt perturbations, which may not capture the full space of adaptive or collusive adversaries active in real markets. The synthetic traces maintain distributional and temporal characteristics of historical market data but naturally abstract away individual strategic idiosyncrasies. While this enables controlled and reproducible measurement, the results should be interpreted as establishing relative effectiveness trends, not exact real-world absolute risk levels.

## 6.3 Implications and Future Work

A next step is grammar-constrained or type-checked decoding that *compiles* to policy-safe actions, reducing validator burden. Typed tool adapters could prevent tool-level jailbreaks by construction.

Automate adversarial data generation and paraphrase search in CI to sustain coverage as policies and prompts evolve. Couple with canary prompts and drift detection to flag regression in defenses.

Explore formal methods to verify output contracts and tool compositions, and to certify privacy maps (e.g., quantization/anonymity guarantees) beyond empirical detectors.

Replicate across base models and domains to test robustness of rankings. Extend to collaborative multi-agent settings where emergent behaviors and collusion may alter the risk surface.

Design escalation policies that integrate operator feedback into the gateway (e.g., semi-automated quarantine and replay), and study long-term learnability and mental load under real on-call schedules.

## 7 Conclusion

This paper presented a systematic study of securing LLM-in-the-loop software in a safety-critical setting. We formalized the problem and derived bounds linking guardrails to residual risk, leakage, and latency (§4); operationalized them as a Security Gateway with four families of controls (input filtering, retrieval filters, constrained decoding, output contracts); and evaluated their impact using a market-scale case with a red-team harness, telemetry, and reproducible profiles (§5).

Empirically, we mapped the baseline risk surface (RQ1, §5.1), then quantified mitigation effects (RQ2, §5.2) and the utility/robustness trade-offs (RQ3, §5.3), and finally assessed operator costs (RQ4, §5.4). Across attack classes and injection points, *combined input filtering + constrained decoding* reduced attack success by **38–62%** with  $\leq 7\%$  utility loss, while *privacy-aware formatting + output contracts* roughly **halved leakage** with marginal overhead. The ablation analysis (§5.5) showed strong marginal effects for input filtering and output contracts and complementary interactions when paired with decoding constraints; latency penalties were bounded and aligned with our theoretical acceptance/latency bounds.

For practice, our results support a staged deployment: (i) enable input filtering and output contracts as a default posture; (ii) add retrieval allow/deny lists whenever retrieval/tool use is present; and (iii) tighten decoding under higher-risk scenarios, guided by profile selection under utility/latency budgets. The signed, hash-chained audit trail and schema-checked outputs make incidents explainable and runs reproducible, offering a practical path to Secure DevOps for LLM applications.

Future work includes typed/tool-safe decoding, adaptive red-teaming in CI, and cross-model, cross-domain studies. Overall, our blueprint provides actionable, evidence-backed guidance for building trustworthy, auditable, and privacy-aware LLM-enabled software.

**Author Contributions** Wenting Wang led the overall conceptualization of the framework, designed the system architecture, and contributed to writing a significant part of the manuscript. Jing Liu contributed to the implementation of privacy-preserving computation techniques and conducted simulation experiments using both synthetic and real-world datasets. Bruce Gu, Changwei Wang, and Youyang Neil Qu were responsible for building the multi-agent simulation platform and integrating searchable encryption into the data processing pipeline. They also helped write the evaluation part. Dongqin Feng supervised the technical aspects related to security methods and agent coordination logic, and assisted in interpreting the experimental results. All authors reviewed and approved the final manuscript.

**Funding** This work is supported by the science and technology project of State Grid Shandong Electric Power Company “Research and Implementation of Key Technologies for Collaborative Defense of Network Security in Dispatch Automation System” (52062624000C).

**Data Availability** A publicly available, open-sourced dataset is used.

## Declarations

**Ethical Approval** Not applicable.

**Informed Consent** Not applicable.

**Conflicts of Interest** No conflict of interest.

**Clinical Trial Number** Not applicable.

## References

- Lu X, Qiu J, Yang Y, Zhang C, Lin J, An S (2024) IEEE Trans Energy Mark Policy Regul
- Pourdaryaei A, Mohammadi M, Mubarak H, Abdellatif A, Karimi M, Gryazina E, Terzija V (2024) Expert Syst Appl 235:121207
- Jiang W, Wu C (2024) IEEE Transactions on Energy Markets. Policy and Regulation 2(3):339
- Zheng J, Liang ZT, Li Y, Li Z, Wu QH (2024) IEEE Trans Industr Inf 20(4):6582
- Wei X, Xu Y, Sun H, Chan WK (2024) IEEE Trans Smart Grid
- Singh G, Bali KK (2024) In: 2024 international joint conference on neural networks (IJCNN) (IEEE, 2024), pp 1–7
- Stegman J, Trottier PJ, Hillier C, Khan H, Mannan M (2023) In: 32nd USENIX security symposium (USENIX Security 23), pp 3583–3600
- Zhan J, Zhang W, Zhang Z, Xue H, Zhang Y, Wu Y (2025) In: Proceedings of the AAAI conference on artificial intelligence, vol 39 (2025), vol. 39, pp 1022–1030
- Feng X, Zhu X, Han QL, Zhou W, Wen S, Xiang Y (2022) IEEE/CAA J Autom Sinica 10(1):25
- Zhu X, Zhou W, Han QL, Ma W, Wen S, Xiang Y (2025) IEEE/CAA J Autom Sinica 12(2):317. <https://doi.org/10.1109/JAS.2024.124971>
- OpenAI (2023) Gpt-4 technical report. <https://openai.com/research/gpt-4>. Accessed April 2024
- Zhou W, Zhu X, Han QL, Li L, Chen X, Wen S, Xiang Y (2025) IEEE/CAA J Autom Sinica 12(1):1. <https://doi.org/10.1109/JAS.2024.124983>
- Chowdhery A, Narang S, Devlin J et al (2022) In: Proceedings of the international conference on machine learning (ICML). [arXiv:2204.02311](https://arxiv.org/abs/2204.02311)
- Deng Z, Ma W, Han Q, Zhou W, Zhu X, Wen S, Xiang Y (2025) IEEE/CAA J Autom Sinica 12(5):872–893. <https://doi.org/10.1109/JAS.2025.125498>
- Zhou H, Hu C, Yuan Y, Cui Y, Jin Y, Chen C, Wu H, Yuan D, Jiang L, Wu D et al (2024) IEEE Commun Surv Tutor
- Fakhoury S, Naik A, Sakkas G, Chakraborty S, Lahiri SK (2024) IEEE Trans Softw Eng
- Li M, Zhao S, Wang Q, Wang K, Zhou Y, Srivastava S, Gokmen C, Lee T, Li EL, Zhang R et al (2024) Adv Neural Inf Process Syst 37:100428
- Li Z, Zhang R, Wang Z, Xie Z, Song Y (2025) Neurocomputing p 130105
- Deng Z, Guo Y, Han C, Ma W, Xiong J, Wen S, Xiang Y (2025) ACM Computing Survey 57(7). <https://doi.org/10.1145/3716628>
- Gough MB, Santos SF, AlSkaif T, Javadi MS, Castro R, Catalão JP (2021) IEEE Trans Industr Inf 18(1):707
- Cheng Z, Ye F, Cao X, Chow MY (2021) IEEE Transactions on Smart Grid 12(6):5233
- Zheng Z, Wang T, Bashir AK, Alazab M, Mumtaz S, Wang X (2021) IEEE Trans Comput 71(11):2915
- Zhou X, Wang B, Guo Q, Sun H, Pan Z, Tian N (2023) IEEE Trans Power Syst 39(1):602
- Wang D, Wu P, Li B, Du H, Luo M (2022) Electr Power Syst Res 212:108223
- He J, Treude C, Lo D (2024) ACM Trans Softw Eng Methodol
- Walliser JC, de Visser EJ, Shaw TH (2023) Comput Hum Behav 143:107671
- Siu HC, Peña J, Chen E, Zhou Y, Lopez V, Palko K, Chang K, Allen R (2021) Adv Neural Inf Process Syst 34:16183
- Wang JJ, Wang L (2021) IEEE Trans Evol Comput 26(3):461
- Saggio V, Asenbeck BE, Hamann A, Strömberg T, Schiansky P, Dunjko V, Friis N, Harris NC, Hochberg M, Englund D et al (2021) Nature 591(7849):229
- Zhang Z, Yao Y, Zhang A, Tang X, Ma X, He Z, Wang Y et al (2025) ACM Comput Surv 57(8):1
- Amirkhani A, Barshooi AH (2022) Artif Intell Rev 55(5):3897

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

## Authors and Affiliations

Wenting Wang<sup>1,2</sup> · Jing Liu<sup>2</sup> · Bruce Gu<sup>3,4</sup> · Changwei Wang<sup>3,4</sup> · Y. Neil Qu<sup>3,4</sup> · Dongqin Feng<sup>5</sup>

✉ Y. Neil Qu  
quyy@sdas.org

✉ Dongqin Feng  
fengdq@zju.edu.cn

Wenting Wang  
12232093@zju.edu.cn; wenting\_research@163.com

Jing Liu  
liujing\_sddl@163.com

Bruce Gu  
gusj@sdas.org

Changwei Wang  
wangcw@sdas.org

<sup>1</sup> State Key Lab. of Industrial Control Technology, Zhejiang University, Hangzhou 310027, China

<sup>2</sup> State Grid Shandong Electric Power Research Institute, Jinan 250003, China

<sup>3</sup> Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu, China

<sup>4</sup> Shandong Provincial Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China

<sup>5</sup> College of control science and engineering, Zhejiang University, 310027 Hangzhou, China